第七章 AI 模块设计

AI 模块是 AI Workbench 的核心竞争力所在，位于 `backend/app/ai/`。它是一组供服务层调用的库：**AI 模块从不反向调用 API 层，也不直接访问数据库**——所有持久化由服务层完成。本章描述提供商抽象与降级策略、离线启发式引擎、提示词版本化、结构化输出、RAG 检索管线、工具调用智能体、Token/延迟/成本核算、重试策略与执行追踪。

7.1 设计目标

AI 模块的设计围绕四条原则：

1. **提供商无关**：业务代码只依赖抽象接口，切换 OpenAI 兼容服务、本地 Ollama 或离线引擎都不改业务逻辑。
2. **零依赖可运行**：在没有 API Key、没有外部服务的环境下，系统仍能完成问答、摘要、规划、智能体等全部工作流。
3. **可追溯**：每次 AI 调用都落库，记录提供商、模型、提示词版本、检索结果、工具调用、Token 与成本。
4. **不信任模型输出**：模型返回的任何内容都先归一化、再校验，才允许进入业务。

7.2 提供商抽象

`app/ai/providers/base.py` 定义两个抽象接口，它们是业务代码唯一依赖的东西：

```python
class LLMProvider(ABC):
name: str
@property
def supports_tool_calling(self) -> bool: return False
@abstractmethod
async def chat(self, messages, *, temperature=0.2, max_tokens=2048,
tools=None, json_mode=False) -> ChatResult: ...
async def aclose(self) -> None: ...

class EmbeddingProvider(ABC):
name: str
@property
@abstractmethod
def dimension(self) -> int: ...
@abstractmethod
async def embed(self, texts) -> EmbeddingResult: ...
async def aclose(self) -> None: ...
```

与提供商无关的值对象定义在 `app/ai/types.py`：`ChatMessage`、`ToolCallRequest`、`Usage`、`ChatResult`、`EmbeddingResult`、`RetrievedChunk`、`ToolSpec`、`ToolResult`。**除 `app/ai/providers/` 之外，任何代码都不知道当前使用哪个厂商。**

7.2.1 具体提供商

类	模块	说明
`OpenAICompatibleProvider`	`openai_compatible.py`	任意 OpenAI 兼容 `/chat/completions`（DeepSeek、Qwen、OpenAI、SiliconFlow 等）
`OpenAICompatibleEmbeddingProvider`	`openai_compatible.py`	`/embeddings`，维度可从首次响应探测
`OllamaProvider`	`ollama.py`	完全本地，提供 `list_models()`
`OllamaEmbeddingProvider`	`ollama.py`	本地嵌入
`HeuristicLLMProvider`	`heuristic.py`	确定性离线引擎（兜底）
`HeuristicEmbeddingProvider`	`heuristic.py`	确定性哈希 n-gram 嵌入器（兜底）

7.3 工厂与优雅降级

`app/ai/providers/factory.py` 是"配置变为实例"的唯一位置，并按"类型 + 模型 + base URL"派生的键缓存提供商。

7.3.1 `build_llm_provider()`

```
AI_PROVIDER == "openai_compatible" 且 AI_API_KEY 非空
→ OpenAICompatibleProvider(kind="openai_compatible")

AI_PROVIDER == "ollama"
→ OllamaProvider(base_url 或 http://127.0.0.1:11434)

否则（openai_compatible 但 Key 为空，或 AI_PROVIDER == "offline"）
→ HeuristicLLMProvider
ProviderInfo(kind="offline", model="heuristic-extractive-v1",
degraded=True, reason=...)
```

记录的 `reason` 为以下之一：

- `"AI_API_KEY is empty; using the offline heuristic engine."`
- `"Ollama was selected but the provider is unavailable; using offline engine."`
- `"AI_PROVIDER=offline explicitly selected."`

7.3.2 `build_embedding_provider()`

形状相同，用 `EMBEDDING_PROVIDER` 选择。嵌入的 base URL 与 API Key 通过 `settings.resolved_embedding_base_url()` / `resolved_embedding_api_key()` 回退到对话配置。离线兜底为 `HeuristicEmbeddingProvider`（`model="heuristic-hash-ngram-v1"`，`degraded=True`）。

7.3.3 降级状态的可见性

降级不是隐藏状态，而是显式暴露：

- `GET /api/v1/health/deps` 返回 `ai.llm.degraded`、`ai.llm.reason`、`ai.embedding.degraded` 以及顶层布尔 `ai_degraded`；

- `GET /api/v1/stats/dashboard` 返回 `providers.llm` / `providers.embedding`（各含 `{kind, model, degraded, reason}`）；
- 每条 `ai_runs` 记录实际应答的 `provider` 与 `model`。

系统**从不假装离线引擎是云端模型**：未配置 Key 时，凡是显示模型信息的地方，`provider` 都是 `"offline"`、`model` 都是 `"heuristic-extractive-v1"`。

7.4 离线启发式引擎

`app/ai/providers/heuristic.py` 是**真实实现，而非占位桩**。正是它使全部工作流在零外部依赖、零 API Key 的条件下可用。

7.4.1 分词

`tokenize(text)` 与语言无关：

- 拉丁词与数字按 `[A-Za-z][A-Za-z0-9_+#.-]*|\d+` 提取并小写；
- CJK 连续段（`[\u4e00-\u9fff]+`）扩展为**字符二元组**，同时保留整段用于精确短语匹配；
- 去除停用词表（中英兼顾），丢弃单字符非 CJK 词元。

7.4.2 嵌入

`HeuristicEmbeddingProvider`（`DEFAULT_DIMENSION = 384`）：

1. 对文本分词；
2. 统计词频；
3. 每个词元用 BLAKE2b（8 字节摘要）哈希到桶 `value % dim`，符号位取自哈希最高位；
4. 累加 `sign * (1 + ln(tf))` ——次线性 TF，抑制重复样板词的影响；
5. 对向量做 L2 归一化。

性质（在 `backend/tests/unit/test_heuristic_provider.py` 中断言）：确定性（同输入同向量）、L2 归一化（模约 1.0）、固定 384 维、语义相关文本的余弦相似度高于无关文本、空输入返回空向量列表。这是一种哈希词袋 n-gram 向量化器：足以支撑词汇—语义召回，无依赖，且永不下载模型。

7.4.3 任务路由

`HeuristicLLMProvider.chat()` 检查系统提示词中的 `<!-- task: <name> -->` 标记（每个内置模板都带），路由到确定性算法；无标记时从用户文本推断（`plan`/`计划` → 规划器，`summar`/`总结` → 摘要，否则 RAG 问答），若提供了工具则使用智能体策略。

任务标记	方法	输出
`rag_answer`	`_answer`	抽取式句子打分答案
`document_summary`	`_summary`	抽取式摘要 + 关键词
`knowledge_extraction`	`_extract`	JSON：摘要、概念、要点、术语、待办、大纲
`task_planner`	`_plan`	JSON：分阶段任务树

`study_plan`	`_study_plan`	JSON：逐日学习计划
`interview_generator`	`_interview`	JSON：问题及参考答案
`note_generation`	`_note`	结构化 Markdown 笔记
`query_rewrite`	`_rewrite`	JSON：2 – 3 个查询变体
`agent_system`	`_agent_step`	每步一个确定性工具调用

7.4.4 抽取式问答

句子切分后过滤长度大于 8 者，按下式打分：

```
score = 2.0 * query_term_overlap
+ 0.4 * lexical_density (唯一词/总词数)
+ 0.2 * position_prior (1 / (1 + 0.08 * index))
```

取前五句组成要点式回答，前缀
`根据检索到的资料，可以归纳出以下要点（离线启发式抽取）`，并声明内容仅来自检索
上下文。无上下文时返回诚实的"未找到资料"提示，引导用户上传文档或配置
`AI_API_KEY`。

7.4.5 结构化抽取

- **摘要**（`_summary`）：高频词、句子显著度、位置先验与长度惩罚；返回编号要点与
关键词表。
- **知识抽取**（`_extract`）：概念取自高频多字词；术语配其定义句；待办由
`TODO|FIXME|待办|下一步|需要|要完成` 正则识别（回退到标题）；大纲取自 Markdown 标题。
- **规划**（`_plan`）：标题（或短句，或目标衍生主题）成为 3 – 5 个阶段，每阶段三个
任务，携带描述、整数工时、优先级与可验证的完成标准。
- **面试题**（`_interview`）：每个显著主题一个问题，以定义句为答案并附追问。
- **学习计划**（`_study_plan`）：把计划摊平为逐日条目。

7.5 RAG 检索管线

RAG 管线位于 `app/ai/rag/`，由 `pipeline.py`（编排）、`retriever.py`（检索与融合）、
`chunker.py`（分块）、`embedding.py`（嵌入服务）组成。十个阶段如下：

```mermaid
flowchart TD
R0["0. Query rewrite"] --> R1["1. Embed query"]
R1 --> R2["2. Dense retrieval (Qdrant)"]
R1 --> R3["3. Sparse retrieval (PostgreSQL + BM25)"]
R2 --> R4["4. Hybrid fusion (HYBRID_ALPHA)"]
R3 --> R4
R4 --> R5["5. Lexical rerank"]
R5 --> R6["6. Deduplicate / diversify"]
R6 --> R7["7. Budgeted context assembly"]
R7 --> R8["8. Generation"]
```

R8 --> R9["9. Citation extraction"]
```

| 阶段 | 实现 |
|---|---|
| 0 查询改写 | `query_rewrite` 提示词产生 2 – 3 个变体 |
| 1 查询嵌入 | `EmbeddingService` 调 `EmbeddingProvider` |
| 2 稠密检索 | Qdrant `search`，按 `knowledge_base_id` 过滤 |
| 3 稀疏检索 | `ILIKE` 取候选 + 进程内 BM25 打分（`Retriever._sparse_search`） |
| 4 混合融合 | 最小—最大归一化加权融合（`Retriever._fuse`） |
| 5 词法重排 | 覆盖率/短语/标题/长度加权 |
| 6 去重与多样化 | 去除近重复分块 |
| 7 预算组装 | 在 Token 预算内拼装上下文 |
| 8 生成 | `LLMProvider.chat` |
| 9 引用抽取 | 映射回 `ai_run_sources` |

### 7.5.1 稀疏腿与 BM25

候选先经 `ILIKE` 从 PostgreSQL 取得，再在进程内用 Okapi BM25 式公式打分（`k1 = 1.5`，`b = 0.75`）：

```
score = Σ_t idf(t) * (tf(t) * (k1 + 1)) / (tf(t) + k1 * (1 - b + b * len/avg_len))
```

7.5.2 混合融合

`_fuse(dense, sparse, alpha)` 对两路分数先做最小—最大归一化再加权：

```
score = alpha * dense_norm + (1 - alpha) * sparse_norm
```

其中 `alpha = HYBRID_ALPHA`（默认 **0.6**，即稠密腿占主导）。

7.5.3 词法重排

在融合分之上叠加词法分：

特征	权重	定义
查询词覆盖	0.45	`|查询词 ∩ 正文词| / |查询词|`
精确短语命中	0.25	查询的小写二元组出现在正文中则为 1.0
标题匹配	0.20	对分块标题的词覆盖
长度先验	0.10	`200 ≤ len ≤ 1600` 为 1.0，否则 0.6

```
lexical = 0.45*coverage + 0.25*phrase + 0.20*heading_match + 0.10*length_prior
score = round(0.65 * fused_score + 0.35 * lexical, 6)
```
```

7.5.4 分块策略

`chunker.py` 采用**标题感知**的分块：先按 Markdown 标题切分为小节，再在小节内按**句边界**打包到目标长度，块间保留**重叠**，并在每块内容前加**标题面包屑前缀**（如 `H1 > H2 > H3`）。面包屑前缀使分块在脱离原文后仍保有上下文，显著提升检索与重排质量。`document_chunks.heading` 单独保存标题，`token_estimate` 保存估算 Token 数。

7.6 工具调用智能体

智能体位于 `app/ai/agent/`，由 `executor.py`（循环）与 `tools.py`（工具注册表）组成。设计规则是**智能体永不直接访问数据库**——所有读写都通过工具，工具内部再经服务层/仓储层完成。

7.6.1 工具清单

`build_default_registry()` 注册 **11 个工具**，`GET /agents/tools` 返回实时目录：

工具	类别	写操作	用途
`knowledge_search`	read	否	知识库内混合检索
`document_read`	read	否	读取文档文本（受 `max_chars` 限制）
`task_list`	read	否	列出已有任务（避免重复）
`project_list`	read	否	列出项目以便归属
`calculator`	utility	否	AST 受限算术
`task_planner`	ai	否	从目标生成分阶段计划（不落库）
`task_create`	write	是	创建单个任务
`task_create_batch`	write	是	从计划/目标创建任务树
`task_update`	write	是	修改状态/优先级/工时/标准
`note_create`	write	是	持久化 Markdown 笔记
`knowledge_link`	write	是	把文档关联到任务

7.6.2 执行循环

```mermaid
stateDiagram-v2
state "组装提示" as Build
state "调用模型" as Call
state "判定" as Decide
state "执行工具" as Exec
state "记录轨迹" as Trace
state "最终答复" as Final
state "达到上限" as Limit
state "循环保护" as LoopGuard
[*] --> Build
Build --> Call
```

Call --> Decide
Decide --> Exec: 有工具调用
Exec --> Trace
Trace --> Call
Decide --> Final: 无工具调用
Decide --> Limit: 步数耗尽
Decide --> LoopGuard: 重复相同调用
Final --> [*]
Limit --> [*]
LoopGuard --> [*]
```

- 每步最多执行**一个**工具；
- 循环终止条件：给出最终答复、耗尽 `max_steps`、检测到**重复相同调用**（循环保护）、或触发写操作上限。

### 7.6.3 安全限制

| 限制 | 取值 | 实现 |
|---|---|---|
| 最大步数 | 默认 8，硬上限 16 | `AgentExecutor.run`（`steps_allowed = max(1, min(max_steps or DEFAULT_MAX_STEPS, 16))`） |
| 写操作上限 | 每次运行 40 | `ToolContext.consume_write` → 抛 `ToolExecutionError` |
| 重复调用保护 | 调用签名哈希 | 记录一条 `SKIPPED` 工具调用，原因 `duplicate call` |
| 工具参数 | 每工具独立校验 | `Tool.run` 先校验再执行 |

`ToolContext` 携带 `session`、`owner_id`、`project_id`、`knowledge_base_ids` 等，并提供 `consume_write()`（超过上限抛 `ToolExecutionError`），从而在框架层面约束智能体的写入行为。

### 7.6.4 为什么工具失败必须可见

一个真实缺陷（缺陷 D5）说明了这一点：服务模块合并后，智能体工具仍引用旧导入路径，导致**写工具静默返回 `ok:false`**——智能体以为写成功了，实际什么都没发生。修复后，工具执行失败会以明确的错误写入 `tool_calls`，并反映在最终答复中。设计原则由此确立：**工具失败绝不能被吞掉**。

## 7.7 提示词版本化

提示词是**数据，不是代码**，存放在数据库（`prompts`、`prompt_versions`），由 `app/ai/prompts/registry.py` 管理。

- `builtin.py` 定义九个种子模板，各含 `name`、`description`、`category`、`task`、`variables`、`json_output` 与 `content`；`PROMPT_INVENTORY` 是种子清单，`TEMPLATES_BY_NAME` 是查找表。
- `render_template()` 用受限正则 `\{([a-z][a-z0-9_]*)\}` 替换 `{name}` 占位符，因此模板内的字面 JSON 花括号是安全的；未知占位符原样保留。
- `ensure_seeded()` 插入尚未存在的内置模板（幂等；启动与测试时调用）。
```

- `get_prompt(session, name)` 返回当前激活版本，无记录时回退内置模板；业务代码
按名字请求，得到 `ResolvedPrompt`。
- `create_version()` / `activate_version()` / `reset_to_builtin()` 管理历史；创建
与当前完全相同的版本会抛 `ConflictError`；内容上限 40000 字符。
- 缓存键 `prompt:<name>:<namespace_version>`，TTL 由 `CACHE_TTL_PROMPT_SECONDS`
（600 秒）控制；每次写入递增命名空间版本，使修改立即可见。

由于解析出的提示词 `name` 与 `version` 会写入 `ai_runs` 行，任何一次 AI 回答都可
复现：给定一条运行记录，即可知道是哪个提示词版本产生的。

7.8 结构化输出处理

`load_json_object(text)`（`rag/pipeline.py`）是一个宽容的 JSON 提取器：

1. 剥离 Markdown 代码围栏；
2. 尝试 `json.loads`；
3. 失败则用 DOTALL 正则找出首个 `{...}` 块并解析；
4. 仍失败则抛 `AIResponseFormatError`，附带 400 字符预览。

所有期望 JSON 的调用方（分析、规划、改写）都经此助手，因此即便模型把 JSON 包在
散文或围栏里也能工作。工具调用参数则由 `agent/executor.py` 的
`parse_tool_arguments(raw)` 做同样的尽力提取。

7.9 Token、延迟与成本核算

`estimate_tokens(text)`（`ai/types.py`）是无依赖启发式
（**刻意不用 `tiktoken`**，因其运行时会下载 BPE 文件）：

```
tokens = max(1, cjk_chars + round(other_chars / 4))
```

`Usage` 携带 `prompt_tokens`、`completion_tokens`、`total_tokens`，并提供
`merged()` 以便多调用运行（智能体步、多动作分析）累加。

`estimate_cost(model, usage)`（`ai/tracing.py`）应用一张小型公开价目表
（美元/百万 Token），对已知模型族返回估算、对未知模型返回 `0.0` 而非猜测。表中
包括 `gpt-4o-mini`、`gpt-4o`、`gpt-4.1-mini`、`deepseek-chat`、
`deepseek-reasoner`、`qwen-plus`、`qwen-turbo`、`text-embedding-3-small`、
`text-embedding-3-large`。

延迟按提供商调用与按运行分别计量，均写入 `ai_runs.latency_ms`，聚合平均值由
`GET /ai-runs/metrics` 与仪表盘暴露。

7.10 重试、超时与退避

`app/core/retry.py` 是手写的有界重试助手（**不用 `tenacity`**），使策略显式、可测、
在 HTTP/AI/worker 各处一致。`RetryPolicy` 默认：
`max_attempts=3`、`base_delay=1.0`、`max_delay=20.0`、`multiplier=2.0`、
`jitter=0.25`、`max_elapsed=180.0`。

`retry_async()` 强制规则：

- 只重试显式标记 `retryable` 的错误；
- `asyncio.CancelledError` 永不重试；
- 尝试次数硬上限；
- 总墙钟时间受 `max_elapsed` 约束；
- 每次重试都记录日志（`retry_scheduled`）；
- 放弃前记录日志（`retry_giving_up`）再抛出。

退避公式：`delay_for(attempt) = min(base_delay * multiplier**(attempt-1), max_delay)`，并加对称抖动。超时设置：对话 `AI_TIMEOUT_SECONDS`（90 秒），嵌入 `EMBEDDING_TIMEOUT_SECONDS`（60 秒），worker 另加任务级 `asyncio.timeout(WORKER_JOB_TIMEOUT_SECONDS)`（900 秒）。

7.11 错误分类

`app/core/errors.py` 中 `AIError.retryable` 驱动重试策略：

异常	代码	HTTP	可重试
`AIError`	`ai_error`	502	否
`AIProviderError`	`ai_provider_error`	502	是
`AITimeoutError`	`ai_timeout`	502	是
`AIRateLimitError`	`ai_rate_limited`	429	是
`AIAuthError`	`ai_auth_error`	502	否
`AIConfigurationError`	`ai_configuration_error`	502	否
`AIResponseFormatError`	`ai_response_format_error`	502	是
`DocumentParseError`	`document_parse_error`	422	—
`ToolExecutionError`	`tool_execution_error`	502	—
`VectorStoreError`	`vector_store_error`	503	—

7.12 执行追踪

`app/ai/tracing.py` 定义 `RunRecorder`，采用显式的 `start → mutate → finish` 句柄（**而非装饰器**），使运行能承受部分失败：若智能体在第 4 步超时，前三次工具调用仍被记录。

`RunRecorder.start()` 插入一条 `RUNNING` 的 `ai_runs` 行并设置 `run_id` 上下文变量。变更方法：`record_usage`、`record_retry`、`set_retrieval`、`set_result_payload`、`add_sources`（写 `ai_run_sources`）、`record_tool_call`（写 `tool_calls` 并追加到 `tools_used`）、`attach_agent_run`（创建 1:1 的 `agent_runs` 行）。

`finish()` 设置状态、响应文本（上限 60000 字符）、延迟、重试次数、错误码/消息、Token 总量、`cost_estimate` 与 `finished_at`，随后 flush 并清除上下文变量；它具备幂等性（`_finished` 守卫）。大载荷由 `_sanitize()` 裁剪（最大深度 6、列表最多 200 项、字符串最多 8000 字符），确保单条运行记录不会撑爆数据库。

`GET /api/v1/ai-runs/{run_id}` 返回 `run`、`tool_calls`、`sources`、`agent_run`。

`run_type` ∈ `RAG_ANSWER | DOCUMENT_SUMMARY | KNOWLEDGE_EXTRACTION | TASK_PLANNER |
AGENT | NOTE_GENERATION | INTERVIEW_GENERATOR | STUDY_PLAN | EMBEDDING | QUERY_REWRITE`；`status` ∈ `RUNNING | SUCCEEDED | FAILED | TIMEOUT | CANCELLED`。

7.13 本章小结

本章给出 AI 模块的完整设计：以抽象接口隔离提供商、以工厂实现优雅降级并把降级状态显式暴露、以真实可用的离线启发式引擎保证零依赖可运行；RAG 管线通过稠密+稀疏混合检索、加权融合与词法重排提升召回与排序质量，并以标题感知分块保留上下文；工具调用智能体在有界步数、写操作上限与循环保护的约束下运行，且工具失败必须可见；提示词以数据形式版本化，使每次回答可复现；Token/延迟/成本核算与执行追踪为可审计性提供支撑。第 9 章将给出这些机制在真实环境中的验证结果。